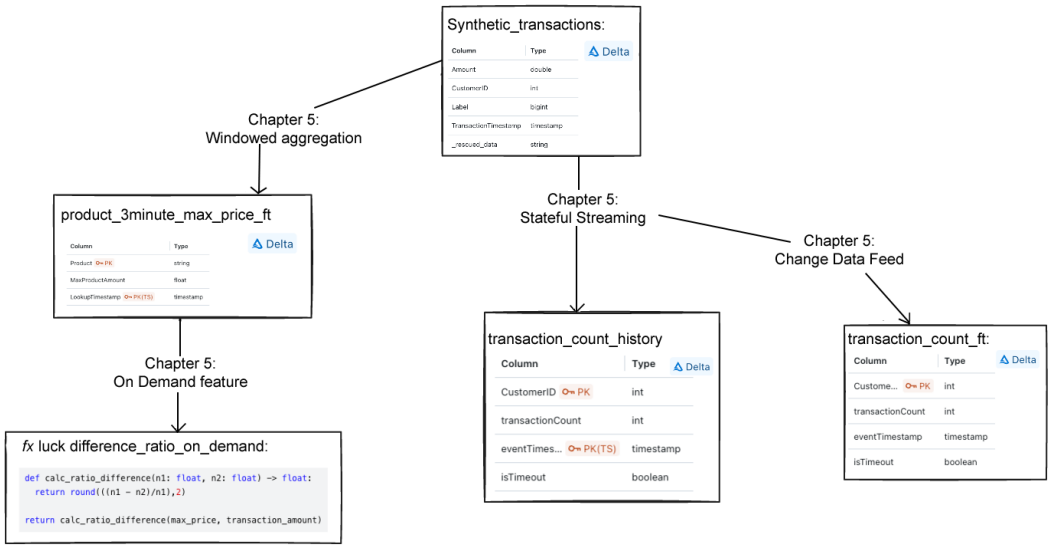


Figure 5.5 – The project pipeline for the Streaming Transactions project

We are building a streaming pipeline to process the incoming transactions into a feature. We will use stateful streaming to count the number of transactions per customer over a two-minute timeframe. Stateful streaming is required because the calculations in the stream need to know how many transactions have already occurred and when a transaction falls outside the two-minute window. This information is known as the state of a customer.

The streaming feature and the on-demand feature are created in two different notebooks. In the code repository, you have the following four notebooks:

- `CH5-01-Generating Records`: This notebook is nearly identical to the data generator used in the previous chapters. The two key differences are that there is now always a product present in the records, and the total (meaning the number of time steps) has been increased to provide a stream for a longer period of time.
- `CH5-02-Auto Loader`: The only change to the `Auto Loader` notebook is the location the data is being written to.
- `CH5-03-FE Using Spark Structured Streaming`: This notebook is explained in detail in the *Building a streaming feature with Spark Structured Streaming* subsection of this project. The code is written in Scala. Stateful streaming is now also available with PySpark. See the *Further reading* section at the end of this chapter for more details.
- `CH5-04-Building Maximum Price Feature Table`: This notebook calculates the maximum price for a product over a time window. The calculated price will be used at inference time by the Python UDF also created in this notebook.